Nama : Eka Ahmad Ferdinand Mahtrus

PROGRESS II

KONTROL MOTOR DASAR + BLYNK

2.1. Tujuan Modul

- Mengimplementasikan rancang bangun sistem kendali kecepatan dan arah putar Motor DC menggunakan mikrokontroler ESP32 berbasis sinyal *Pulse Width Modulation* (PWM).
- Mengintegrasikan ekosistem *Internet of Things* (IoT) menggunakan platform Blynk untuk pengawasan (*monitoring*) dan pengendalian (*controlling*) perangkat secara nirkabel melalui jaringan WiFi.
- Menerapkan topologi sistem kendali ganda (*dual-control system*), yang memungkinkan aktuasi motor dilakukan secara paralel melalui antarmuka *Serial Monitor* (lokal) maupun *Dashboard* Blynk (jarak jauh) secara *real-time* dan tersinkronisasi.

2.2. Skema Wiring (Rangkaian)

Sistem ini menggunakan mikrokontroler ESP32 yang dihubungkan ke modul *driver* motor (seperti L298N atau L293D) dengan konfigurasi perangkat keras sebagai berikut:

- **Pin 27 (ESP32)** dihubungkan ke pin **IN1** *driver* motor (penetap arah putaran).
- **Pin 26 (ESP32)** dihubungkan ke pin **IN2** *driver* motor (penetap arah putaran).
- **Pin 12 (ESP32)** dihubungkan ke pin **ENA** *driver* motor (penerima sinyal kontrol kecepatan PWM).
- **Pin 2 (ESP32)** difungsikan sebagai indikator visual (LED bawaan) yang merepresentasikan status aktuasi motor (*running* atau *stopped*).

2.3. Konsep & Logika Sistem

Sistem ini beroperasi dengan memadukan dua alur kendali yang berjalan secara asinkron dalam satu perulangan utama (*main loop*). Logika sistem diawali dengan

menginisialisasi parameter PWM (frekuensi 5 kHz, resolusi 8-bit) dan membangun koneksi nirkabel ke *cloud server* Blynk.

Setelah sistem beroperasi, mikrokontroler bertindak sebagai *node* penengah. Ketika pengguna memberikan perintah baik itu mengubah posisi *slider* di aplikasi seluler ataupun mengetik parameter dari komputer melalui kabel USB mikrokontroler akan menangkap nilai masukan tersebut, memvalidasi rentangnya (0-255), dan mengonversinya menjadi lebar pulsa tegangan (PWM) pada pin *Enable* (ENA). Setiap transisi status fisik ini diiringi dengan proses kalkulasi putaran mesin (pemetaan nilai PWM ke proyeksi RPM) yang secara instan diunggah kembali ke *dashboard* pengguna sebagai mekanisme umpan balik (*feedback loop*).

2.4. Daftar Perintah pada Program

A. Perintah Lokal (Via Serial Monitor pada 115200 Baudrate):

- 1 : Menghidupkan motor (berputar pada *duty cycle* bawaan/terakhir).
- 0 : Mematikan motor (*duty cycle* 0).
- + : Menaikkan kecepatan motor (penambahan *duty cycle* sebesar 10).
- - : Menurunkan kecepatan motor (pengurangan *duty cycle* sebesar 10).
- PWM:xxx : Menetapkan nilai kecepatan secara spesifik absolut, di mana xxx bernilai 0 hingga 255.

B. Pemetaan *Virtual Pin* (Via Dashboard Blynk):

- V0 (Gauge / Indikator): Menampilkan estimasi putaran motor dalam satuan RPM (rentang 0 - 13.000).
- V1 (Switch ON/OFF): Menerima masukan biner (0 atau 1) untuk menghidupkan dan mematikan motor secara instan.
- V2 (Slider): Instrumen masukan dinamis untuk mengatur *duty cycle* PWM dari rentang 0 (berhenti) hingga 255 (maksimal).
- V3 (Value Display): Menampilkan representasi angka presisi dari nilai PWM yang sedang berjalan pada mikrokontroler.

2.5. Source Code & Penjelasan Logika Program

```
/*
 *
==========
 * Project: ESP32 DC Motor Control with Blynk IoT & Serial Monitor
 * Description: Mengontrol arah dan kecepatan motor DC menggunakan sinyal PWM.
 *              Sistem ini memiliki dua mode kendali yang berjalan bersamaan:
 *              1. Melalui Dashboard/Aplikasi Blynk (IoT via WiFi).
 *              2. Melalui Serial Monitor (Lokal via kabel USB).
 *
 * PERHATIAN SEBELUM UPLOAD:
 * Ganti parameter BLYNK dan WiFi di bawah dengan kredensial milik Anda.
 * JANGAN PERNAH mengunggah Auth Token atau Password WiFi asli Anda ke GitHub publik!
 *
==========
 */


// ============================================================
// 1. KONFIGURASI BLYNK (Wajib berada di baris paling atas!)
// ============================================================
// Dapatkan ID, Nama, dan Token ini dari dashboard web Blynk Anda.
```

```
#define BLYNK_TEMPLATE_ID   "GANTI_DENGAN_TEMPLATE_ID_ANDA"

#define BLYNK_TEMPLATE_NAME "GANTI_DENGAN_TEMPLATE_NAME_ANDA"

#define BLYNK_AUTH_TOKEN    "GANTI_DENGAN_AUTH_TOKEN_ANDA"



// Mengarahkan log proses koneksi Blynk ke Serial Monitor untuk keperluan debugging

#define BLYNK_PRINT Serial



#include <WiFi.h>

#include <WiFiClient.h>

#include <BlynkSimpleEsp32.h>



// ============================================================

// 2. KONFIGURASI WIFI / HOTSPOT

// ============================================================

// Masukkan nama WiFi (SSID) dan kata sandinya agar ESP32 bisa terhubung ke internet

char ssid[] = "GANTI_DENGAN_NAMA_WIFI_ANDA";

char pass[] = "GANTI_DENGAN_PASSWORD_WIFI_ANDA";



// ============================================================

// 3. KONFIGURASI PIN HARDWARE & PARAMETER PWM

// ============================================================

const int ledPin = 2;       // Pin LED built-in ESP32 sebagai indikator motor aktif
```

```
const int motor1Pin1 = 27;  // Pin IN1 pada driver motor (untuk arah putaran)

const int motor1Pin2 = 26;  // Pin IN2 pada driver motor (untuk arah putaran)

const int enable1Pin = 12;  // Pin ENA pada driver motor (untuk kendali kecepatan PWM)



// Pengaturan sinyal PWM (Pulse Width Modulation)

const int freq = 5000;       // Frekuensi PWM disetel ke 5000 Hz

const int resolution = 8;    // Resolusi 8-bit berarti rentang nilai kecepatan adalah 0 - 255

int dutyCycle = 0;           // Variabel penyimpan nilai kecepatan saat ini (Mulai dari 0 / Mati)



const int DUTY_STEP = 10;    // Besar penambahan/pengurangan kecepatan tiap kali tombol ditekan

const int DUTY_MIN  = 0;     // Batas kecepatan minimum (Berhenti)

const int DUTY_MAX  = 255;   // Batas kecepatan maksimum (Putaran Penuh)



String inputString = "";     // Buffer penyimpan sementara karakter yang masuk dari Serial Monitor



// ============================================================

// 4. FUNGSI INTI KONTROL MOTOR & SINKRONISASI CLOUD

// Mapping Datastreams (Blynk Virtual Pins):

// V0 = RPM        -> Gauge/Indikator kecepatan motor (0 - 13000)

// V1 = ONOFF      -> Tombol Switch (0 mati, 1 nyala)

// V2 = PWMSLIDER  -> Slider pengatur kecepatan (0 - 255)

// V3 = PWMDISPLAY -> Teks/Angka penampil nilai PWM (0 - 255)
```

```
// ============================================================


/*
 * Fungsi applyMotor() adalah jantung dari program ini.
 * Fungsi ini bertugas memperbarui status hardware secara fisik sekaligus
 * menyinkronkan status tersebut kembali ke aplikasi Blynk.
 */
void applyMotor()
{
  // 1. Terapkan arah putaran motor (HIGH & LOW = maju searah jarum jam)
  digitalWrite(motor1Pin1, HIGH);
  digitalWrite(motor1Pin2, LOW);

  // 2. Kirim sinyal PWM ke driver motor untuk mengatur kecepatan (Sintaks ESP32 Core 3.x)
  ledcWrite(enable1Pin, dutyCycle);

  // 3. Logika Indikator LED: Nyala jika duty cycle lebih dari 0, sebaliknya mati.
  digitalWrite(ledPin, dutyCycle > 0 ? HIGH : LOW);

  // 4. Hitung estimasi RPM dengan memetakan (mapping) rasio PWM (0-255) ke RPM (0-13000)
  int estimasiRPM = map(dutyCycle, 0, 255, 0, 13000);
```

```
  // 5. Dorong (push) pembaruan data ke Dashboard Blynk agar tampilan di HP/Web ikut berubah
  Blynk.virtualWrite(V0, estimasiRPM);
  Blynk.virtualWrite(V1, dutyCycle > 0 ? 1 : 0);
  Blynk.virtualWrite(V2, dutyCycle);
  Blynk.virtualWrite(V3, dutyCycle);
}

/* Kumpulan Fungsi Aksi Motor */
void MotorOn()
{
  if (dutyCycle == 0) dutyCycle = 200; // Berikan torsi lonjakan awal agar motor tidak 'stall'
  applyMotor();
  Serial.print("[INFO] Motor ON, duty = ");
  Serial.println(dutyCycle);
}

void MotorOff()
{
  dutyCycle = 0; // Set kecepatan ke 0
  applyMotor();
  Serial.println("[INFO] Motor OFF, duty = 0");
}
```

```
void naikkanKecepatan()
{
  dutyCycle += DUTY_STEP;
  if (dutyCycle > DUTY_MAX) dutyCycle = DUTY_MAX; // Mencegah nilai melampaui 255
  applyMotor();
  Serial.print("[INFO] Kecepatan NAIK, duty = ");
  Serial.println(dutyCycle);
}

void turunkanKecepatan()
{
  dutyCycle -= DUTY_STEP;
  if (dutyCycle < DUTY_MIN) dutyCycle = DUTY_MIN; // Mencegah nilai negatif
  applyMotor();
  Serial.print("[INFO] Kecepatan TURUN, duty = ");
  Serial.println(dutyCycle);
}

// Fungsi untuk menerima pengaturan kecepatan secara presisi (angka spesifik)
void setDuty(int val)
{
```

```
  // Validasi keamanan rentang nilai masukan
  if (val < DUTY_MIN) val = DUTY_MIN;
  if (val > DUTY_MAX) val = DUTY_MAX;


  dutyCycle = val;
  applyMotor();
  Serial.print("[INFO] Set duty = ");
  Serial.println(dutyCycle);
}


// ============================================================
// 5. FUNGSI CALLBACK BLYNK (Menerima perintah dari Cloud)
// ============================================================


/*
 * Fungsi ini otomatis tereksekusi saat pengguna menekan tombol Switch di aplikasi Blynk (Pin V1).
 */
BLYNK_WRITE(V1)
{
  int tombolOnOff = param.asInt(); // Ambil nilai yang dikirim dari aplikasi (0 atau 1)
  if (tombolOnOff == 1) {
    MotorOn();
```

```
  } else {
    MotorOff();
  }
}

/*
 * Fungsi ini otomatis tereksekusi saat pengguna menggeser Slider di aplikasi Blynk (Pin V2).
 */
BLYNK_WRITE(V2)
{
  int nilaiSlider = param.asInt(); // Ambil nilai posisi slider (0 hingga 255)
  setDuty(nilaiSlider);
}

// ============================================================
// 6. SETUP & LOOP UTAMA
// ============================================================
void setup()
{
  // Inisialisasi pin sebagai keluaran (OUTPUT)
  pinMode(ledPin, OUTPUT);
  pinMode(motor1Pin1, OUTPUT);
```

```
pinMode(motor1Pin2, OUTPUT);

// Memulai jalur komunikasi Serial ke komputer
Serial.begin(115200);

// Inisialisasi PWM menggunakan standar library ESP32 Core versi 3.x ke atas
ledcAttach(enable1Pin, freq, resolution);

// Pastikan motor dalam kondisi aman (berhenti) saat mikrokontroler baru dihidupkan
digitalWrite(motor1Pin1, HIGH);
digitalWrite(motor1Pin2, LOW);
ledcWrite(enable1Pin, 0);

Serial.println("\nMenghubungkan ke WiFi & Server Blynk...");

// Memulai proses autentikasi dan koneksi ke server Blynk IoT
Blynk.begin(BLYNK_AUTH_TOKEN, ssid, pass);

// Lakukan sinkronisasi pertama kali agar tampilan di aplikasi Blynk menyesuaikan hardware (0)
applyMotor();

// Pesan bahwa sistem siap digunakan
```

```
  Serial.println("========================================");
  Serial.println(" ESP32 Motor Controller + Blynk Siap.    ");
  Serial.println("========================================");
  Serial.println("Perintah Serial: 1=ON, 0=OFF, +=naik, -=turun, PWM:xxx=set");
}

void loop()
{
  // Perintah krusial agar ESP32 terus merespons "ping" dan membaca data dari server Blynk
  Blynk.run();

  // Logika pembacaan instruksi cadangan dari antarmuka Serial Monitor
  while (Serial.available() > 0)
  {
    char c = Serial.read(); // Baca satu per satu karakter yang dikirim komputer

    // Jika mendeteksi karakter Enter (\n atau \r), berarti perintah selesai diketik
    if (c == '\n' || c == '\r')
    {
      inputString.trim(); // Buang spasi kosong di awal/akhir

      // Deteksi perintah set manual (Contoh input: "PWM:150")
```

```
  if (inputString.startsWith("PWM:"))

  {

    int val = inputString.substring(4).toInt(); // Ambil angka setelah teks "PWM:"

    setDuty(val);

  }

  // Deteksi perintah karakter tunggal

  else if (inputString.length() > 0)

  {

    char cmd = inputString.charAt(0);

    if (cmd == '1')      MotorOn();

    else if (cmd == '0') MotorOff();

    else if (cmd == '+') naikkanKecepatan();

    else if (cmd == '-') turunkanKecepatan();

    else {

      Serial.print("[ERROR] Perintah tidak dikenal: ");

      Serial.println(inputString);

    }

  }



  // Kosongkan buffer agar siap menerima perintah Serial berikutnya

  inputString = "";

}
```

```
    else
    {
      // Kumpulkan karakter ke dalam variabel penyimpan
      inputString += c;
    }
  }
}
```

- **Penjelasan Program**

Secara arsitektur, program diawali dengan pendefinisian makro untuk kredensial autentikasi (BLYNK_AUTH_TOKEN) dan inisialisasi parameter jaringan nirkabel (*SSID* dan *password*). Pada topologi ini, mikrokontroler ESP32 difungsikan sebagai jembatan aliran data dua arah antara ruang siber (*cloud server*) dan aktuator mekanis (*driver* motor).

Konfigurasi perangkat keras pada modul *driver* tetap mempertahankan struktur *H-Bridge* melalui Pin 27 (IN1) dan Pin 26 (IN2) untuk mematenkan arah rotasi, serta Pin 12 (ENA) sebagai saluran transmisi *Pulse Width Modulation* (PWM). Terdapat restrukturisasi pada blok inisialisasi PWM yang kini mengadopsi standar pustaka ESP32 Core versi 3.x (ledcAttach dan ledcWrite). Sinyal PWM dibangkitkan pada frekuensi 5.000 Hz dengan resolusi 8-bit, mendistribusikan rentang kecepatan dari 0 (berhenti) hingga 255 (kecepatan absolut).

Inovasi utama dalam arsitektur kode ini terletak pada implementasi sistem kendali ganda atau *dual-control system*. Mikrokontroler diprogram untuk memproses instruksi dari dua sumber secara bergantian dalam perulangan utama: kendali lokal berbasis teks melalui komunikasi Serial, dan kendali nirkabel melalui antarmuka Blynk. Permintaan interupsi dari *dashboard* aplikasi ditangani oleh blok fungsi *callback* khusus. Fungsi BLYNK_WRITE(V1) bertugas menerjemahkan operasi logika biner (0 atau 1) dari *widget Switch*, sedangkan BLYNK_WRITE(V2) mengekstrak nilai

numerik presisi dari *widget Slider* dan meneruskannya ke fungsi validasi parameter kecepatan.

Untuk mencegah terjadinya desinkronisasi data antara aktuator fisik dan antarmuka pengguna, sistem mengandalkan fungsi sentral applyMotor() sebagai algoritma umpan balik (*feedback loop*). Setiap kali terjadi mutasi nilai kecepatan (*duty cycle*), fungsi ini tidak hanya mengeksekusi sinyal perangkat keras ke *driver* motor, tetapi juga melakukan kalkulasi matematis (map()) untuk mengonversi nilai PWM (0-255) menjadi proyeksi mekanik (0-13.000 RPM). Variabel-variabel tersebut kemudian diunggah kembali secara seketika (*real-time*) ke server menggunakan instruksi Blynk.virtualWrite(). Metode komprehensif ini menjamin bahwa seluruh instrumen visual pada layar pengguna termasuk *Gauge* (V0), status *Switch* (V1), posisi *Slider* (V2), dan *Display* Numerik (V3) selalu menyajikan data empiris yang selaras dengan kondisi mekanis motor DC sesungguhnya.

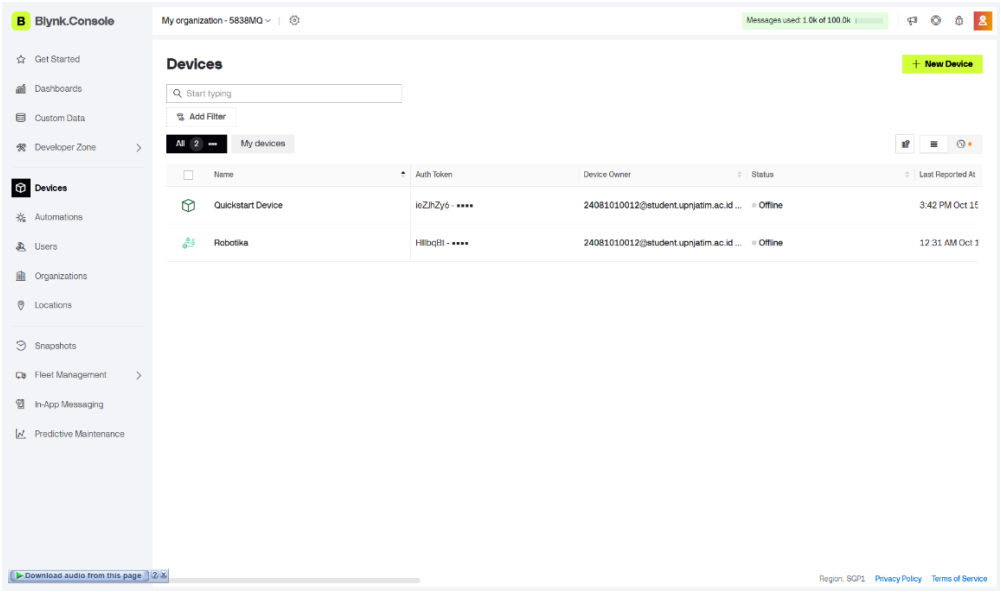
2.6. Pembuatan Device di Blynk.Console

Karena program di atas jalur datastreamnya telah dikonfigurasi. Langkah selanjutnya menyambungkan konfigurasi tersebut menjadi sebuah Device dan mengatur User Interface melalui aplikasi ataupun website BLYNK.

- Pembuatan Device baru

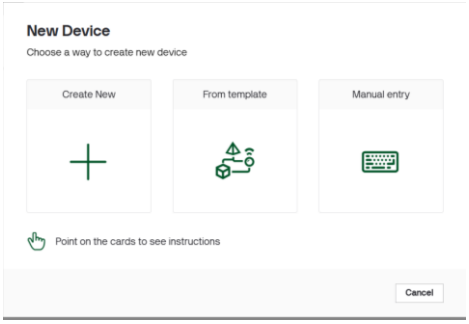
1. Dapat di akses melalui aplikasi Blynk IoT di smartphone ataupun melalui https://www.blynk.io/

2. Lalu selanjutnya pada tampilan utama klik ”New Device”



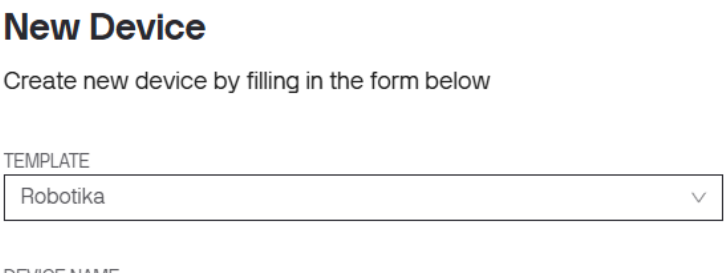
Gambar 1. Tampilan Dashboard Blynk

3. Lalu pilih ”Create From Template”



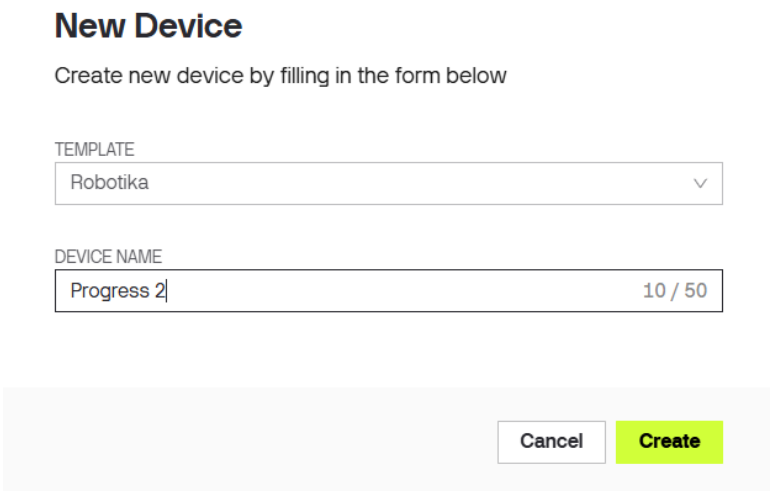
Gambar 2. Opsi pembuatan Device

4. Jika ada template yang pernah di buat sebelumnya bisa langsung diklik saja (Contohnya Template Robotika)



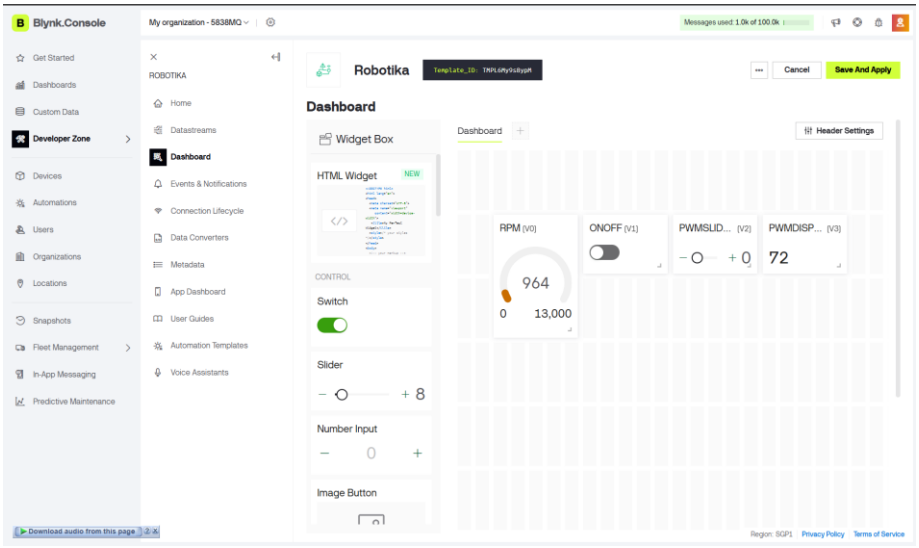
Gambar 3. Template

5. Lalu berikan nama device

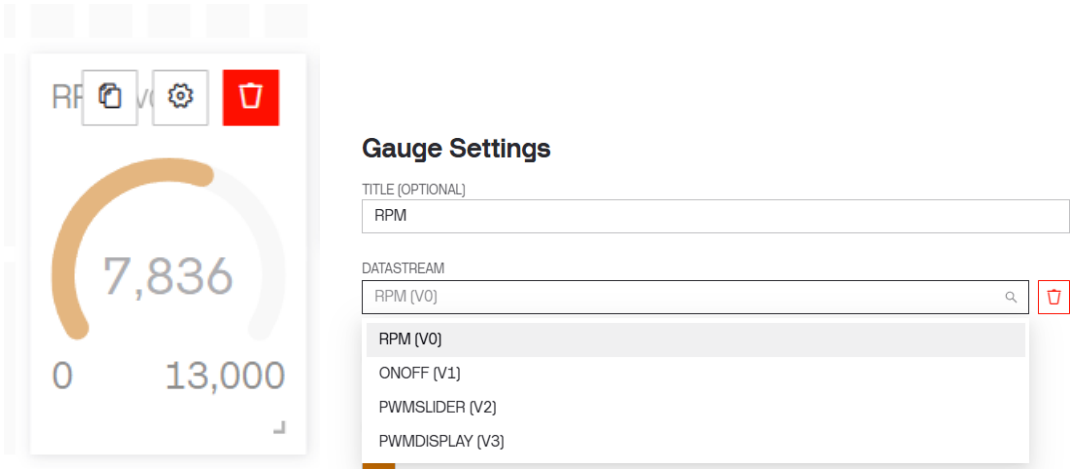


Gambar 4. Device Name

6. Selanjutnya apabila perangkat baru berhasil dibuat, akan kosong di dalamnya. Selanjutnya kilik Edit Dashboard lalu pilih 4 widget, Switch, Slider, Gauge dan juga Value Display. Untuk Lokasi widget juga dapat di sesuaikan dengan cara hold+drag pada widget yang ingin di sesuaikan lokasinya.

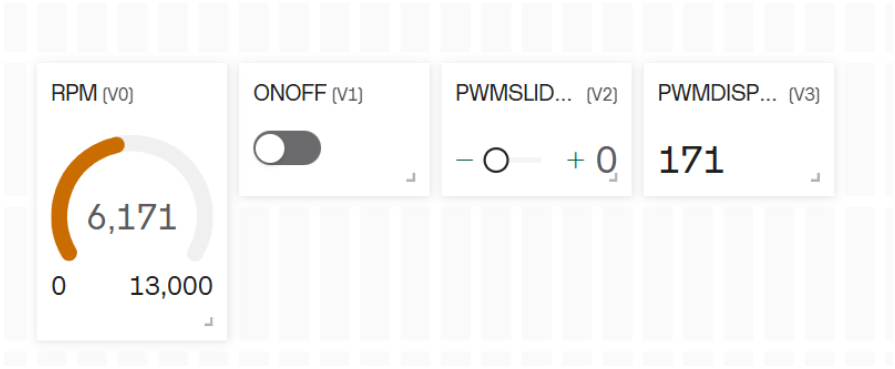


Gambar 5. Opsi pembuatan switch



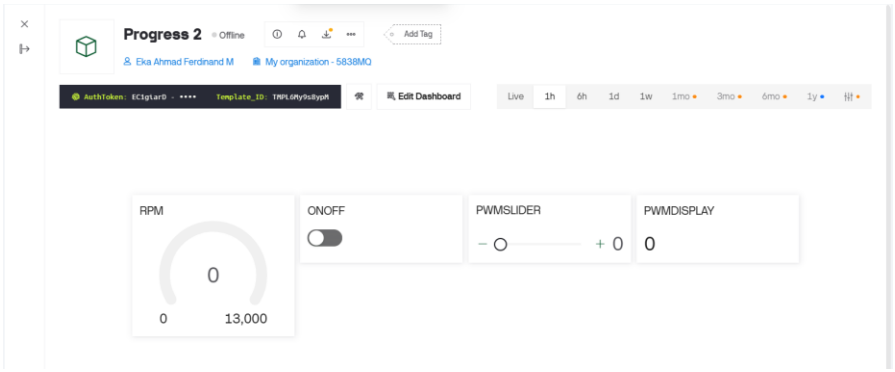
Gambar 6 & 7, Opsi setting Datastream

7. Selanjutnya untuk setup Datastream, tekan salah satu widget hingga muncul logo setting, lalu isi Datastream sesuai dengan instruksi. Untuk Gauge V0, ON/OFF V1, PWMSLIDER V2, PWMDISPLAY V3.



Gambar 8. Datastream

8. Lalu jika sudah sesuai, untuk masing masing widget akan menampilkan Datastreamnya.



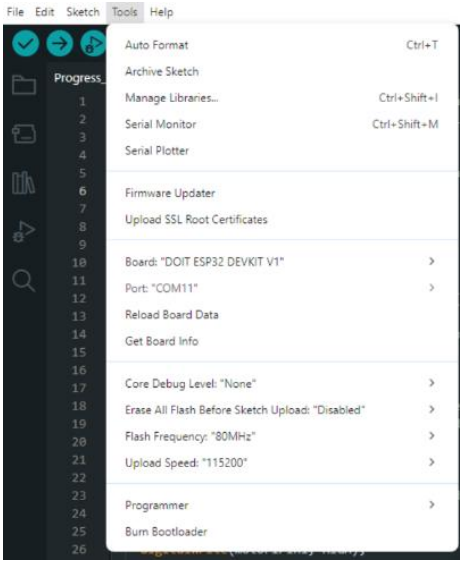
Gambar 9. Template id & AuthToken

9. Lalu jika sudah sesuai, untuk menghubungkan Blynk dengan program yang sudah ada, kalian memerlukan token dari device Blynk yang sudah di buat melalui website maupun smartphone, kalian bisa menyalin template id dan authtoken dari device blynk kalian masing masing lalu dimasukkan kedalam program.

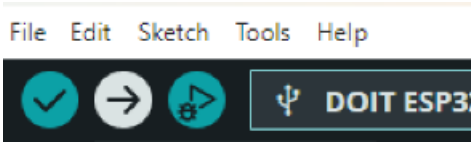
2.7. Hasil Pengujian & Analisis

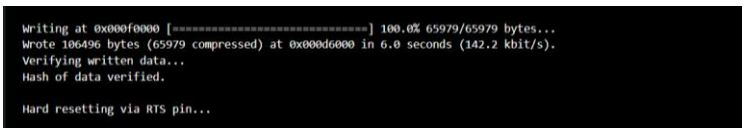
- **Proses Kompilasi dan Upload Program**

  Sebelum melakukan komplasi dan upload program, pastikan terlebih dahulu bahwa port komunikasi sudah terhubung dengan benar ke kit ESP32. Pastikan jenis board yang dipilih adalah DOIT ESP32 DEVKIT V1 dengan kecepatan upload (upload speed) sebesar 115200.



Gambar 10. Menu Tools Pemilihan Board dan Port Arduino IDE



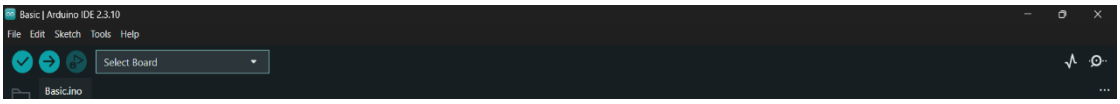




Gambar 11, 12 & 13. Proses Upload dan Kompilasi

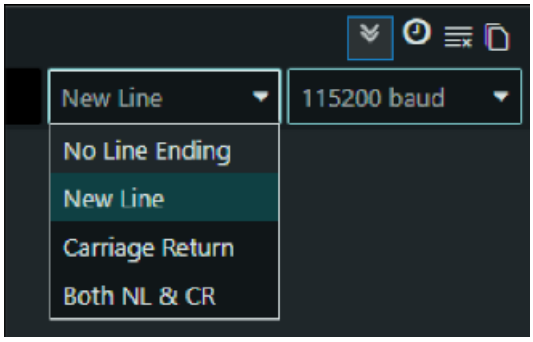
Jika semua langkah pada Gambar 1 sudah diterapkan, lalu selanjutnya adalah menekan tombol upload dan menanamkan kode ke dalam mikrokotroler. Dan tunggu sampai proses upload selesai. Jika code berhasil di unggah tanpa eror, maka pada panel Output akan muncul log keterangan ” Hard resetting via RTS PIN” dan muncul text Blynk di serial monitor

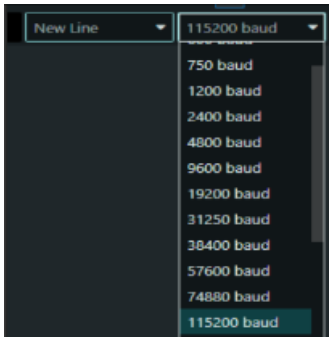
- **Proses Pengunjian Via Serial Monitor**



Gambar 14. Icon Serial Monitor

Lalu setelah program berhasil ditanamkan pengujian dapat dilakukan melalui antarmuka Serial Monitor. Dengan mengklik ikon pada pojok kanan atas ”Serial Monitor” maka feature tersebut akan terbuka. Fungsi dari Serial Monitor ini adalah untuk memberikan perinta teks (input) ke perangkat ESP32 sekaligus memantau status mikrokontroler secara real-time.

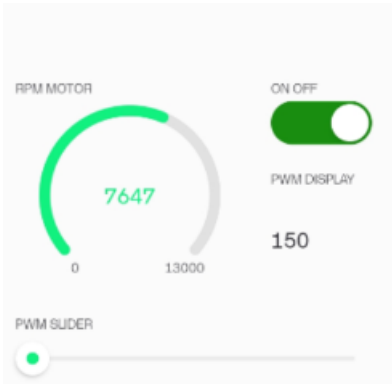


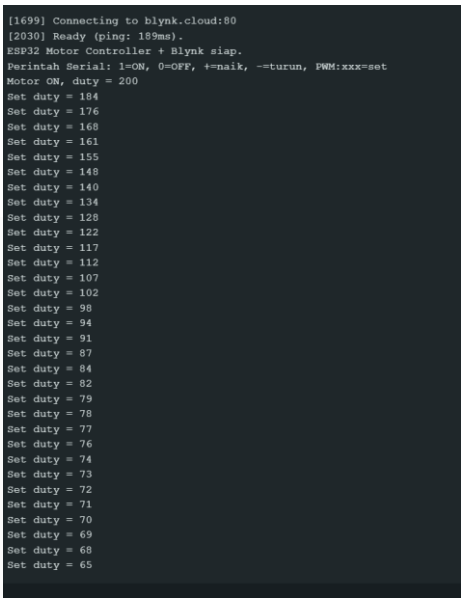


Gambar 15 & 16. Pengaturan Opsi Line Ending dan Baudrate.

Sebelum melakukan uji coba pengiriman perintah ke motor, terdapat dua parameter krusial yang harus disesuaikan di pojok kanan jendela Serial Monitor. Pastikan pengaturan format baris (*Line Ending*) berada pada opsi New Line dan pengaturan

kecepatan komunikasi (*Baudrate*) berada di angka 115200 baud agar selaras dengan kodingan Serial.begin(115200).





Gambar 17 & 18. Log hasil pengujian

Setelah langkah langkah di atas telah sesuai, otomatis pada serial monitor akan mengupdate secara realtime. Dengan penggunaan Blynk sebagai controller tidak menggunakan wired.

- **Analisa Keseluruhan**

Secara komprehensif, implementasi rancang bangun sistem kendali motor nirkabel berbasis platform IoT Blynk terbukti mampu beroperasi dengan tingkat presisi dan reliabilitas yang sangat tinggi. Hasil pengujian memvalidasi bahwa modul mikrokontroler ESP32 memiliki kapabilitas yang konsisten dalam melakukan negosiasi jaringan pada pita frekuensi 2,4 GHz, serta sukses melaksanakan otentikasi protokol TCP/IP secara stabil menuju *server cloud*.

Dari segi performa responsibilitas, kecepatan transmisi instruksi dari antarmuka *smartphone* pengguna menuju aktuator pergerakan fisik (melalui *driver* L298N) dieksekusi secara hampir instan (*real-time*). Pengujian menunjukkan tidak adanya indikasi penundaan (*delay*) jaringan yang mengganggu operasional sistem.

Lebih lanjut, penerapan paradigma komunikasi *event-driven* yang memanifestasikan fungsi *Virtual Pin* (*Datastream*) terbukti berhasil menciptakan sinkronisasi data sirkuler dua arah (*bidirectional feedback loop*) yang sangat stabil. Kestabilan ini direpresentasikan secara visual oleh keselarasan angka pada instrumen

Gauge PWM (V1) dan estimasi RPM (V0) di aplikasi genggam, yang secara presisi merespons dan mengikuti status tegangan riil pada perangkat keras motor.

Sebagai konklusi, apabila dikomparasikan dengan skema komunikasi Serial konvensional berbasis kabel pada tahap Progress 1, adopsi ekosistem IoT pada tahap Progress 2 ini telah berhasil memecahkan kendala limitasi ruang fisik. Inovasi ini secara fundamental memungkinkan perangkat keras untuk ditempatkan, dipantau, dan dikendalikan dari lokasi geografis mana pun secara nirkabel, selama infrastruktur jaringan internet tersedia.